

# VASH vs audit vs VVAH — Benchmark Comparison

**Target:** `datamodel-code-generator` 0.55.0 (Python code-generator library, 50 source files) **Ground truth:** 11 in-version CVEs — `codegen` (6, CWE-94), `SSRF` (3, CWE-918), `path-traversal` (1, CWE-22), `info-leak` (1, CWE-200). (*CVE-2026-55390 excluded as not-in-version.*) **Date:** 2026-07-25

## Methodology (identical across tools — no hints given)

- **Same target** mounted read-only into each tool's own Docker container (execution isolated; PoCs run in-sandbox).
- **Same models:** Opus 4.8 for recon / validate / trace; Sonnet 5 for the rest. (audit's stale model IDs were bumped to match; its pipeline/prompts were left pristine. VVAH ran via its own CLI backend.)
- **Same scorer:** a finding matches a CVE when `class(finding) == class(CVE)` **and** any advisory file-hint is a substring of the finding's file path; greedy 1:1; in-version CVEs only. Class from the finding's CWE when present, else mapped from vuln-class.

## Scoring basis — read this before trusting any number

The only cross-tool-comparable metric is each tool's **delivered output** — the report/SARIF a user actually receives, *after that tool's own dedup/filter*: - VVAH → its SARIF (20 findings, already deduped: 97 raw → 40 FP dropped → 34 dupes merged → 20). - audit / VASH → their `report.json` (13 / 21 findings, after dedup + trace).

A tool's **pre-dedup confirmed** count is **not** comparable across tools, because VVAH's pre-dedup set (its 97 raw candidates) isn't available — only its final 20. So we rank on **delivered** only.

## Scorecard (ranked by delivered recall)

| Metric                                         | VVAH (Visa)                         | audit (evilsocket)   | VASH                                                      |
|------------------------------------------------|-------------------------------------|----------------------|-----------------------------------------------------------|
| Recall — DELIVERED (the ranking metric)        | 4/11 (36%)                          | 2/11 (18%)           | 2/11 (18%)                                                |
| Delivered findings                             | 20                                  | 13                   | 21                                                        |
| Exploit chains                                 | 6                                   | 0                    | 5                                                         |
| Cost                                           | ~7.2M tokens / ~3 hr                | <b>\$48 / 2.9 hr</b> | \$96 / 2.7 hr                                             |
| Coverage                                       | 74 files, 82.2%                     | 50 files             | 50 files                                                  |
| Precision (reported)                           | 20.6% (97→20)                       | —                    | —                                                         |
| Pipeline                                       | agentic SAST, <b>template-aware</b> | lean 8-stage LLM     | 9-stage + deterministic graph engine                      |
| <b>VASH-internal note:</b> pre-dedup confirmed | n/a (unavailable)                   | 4/11                 | 6/11 ( <i>dedup/report drops 4 → delivers 2; see D8</i> ) |

*ai-proofscan reference baseline on this corpus: 6/11.*

## Per-CVE matrix (delivered output — apples-to-apples)

| CVE   | Class   | Hints                             | VVAH | audit | VASH |
|-------|---------|-----------------------------------|------|-------|------|
| 54621 | codegen | template, jinja2, UnionType       | ✓    | .     | .    |
| 54653 | codegen | pydantic_base, dataclass, msgspec | .    | .     | .    |
| 54654 | codegen | template, jinja2, TypeAlias       | .    | .     | .    |

| CVE          | Class     | Hints                        | VVAH        | audit       | VASH        |
|--------------|-----------|------------------------------|-------------|-------------|-------------|
| 54655        | codegen   | jsonschema                   | ✓           | ✓           | ✓           |
| 54656        | codegen   | validator                    | .           | .           | .           |
| 54690        | ssrf      | http                         | ✓           | .           | .           |
| 54691        | ssrf      | http, arguments, <b>main</b> | .           | .           | .           |
| 55415        | codegen   | imports                      | .           | .           | .           |
| 55389        | traversal | jsonschema                   | ✓           | ✓           | ✓           |
| 55391        | ssrf      | http                         | .           | .           | .           |
| 55403        | infoleak  | http                         | .           | .           | .           |
| <b>TOTAL</b> |           |                              | <b>4/11</b> | <b>2/11</b> | <b>2/11</b> |

- **All three deliver:** 54655 (codegen in `jsonschema.py`) and 55389 (traversal in `jsonschema.py`).
- **VVAH uniquely delivers 2:** 54621 (codegen inside `UnionTypeAliasAnnotation.jinja2` — a **template file**) and 54690 (SSRF in `http.py`).
- **Nobody delivers:** 54653/54654/54656/55415 (codegen), 54691/55391 (ssrf), 55403 (infoleak) — 7 of 11 missed by all.

## Per-tool analysis

### VVAH (Visa) — best on this benchmark (4/11 delivered)

20 verified findings (4 critical RCE, 6 high), 6 exploit chains, 82% file coverage. Its decisive edges: it **scans template files** (`.jinja2`) — catching codegen that lives in templates (54621) — and it cleanly surfaces the `http.py` SSRF (54690). Trade-off: lowest precision it reports (20.6%) and highest analysis budget (~3 hr / 7.2M tokens).

### audit (evilsocket) — cheapest, ties VASH on delivery (2/11)

Pure 8-stage LLM hunting, no deterministic engine, no templates. Delivers the 2 easy `.py` CVEs at **half the cost** (\$48). Efficient baseline; leaves recall on the table (no template awareness, no graph coverage).

### VASH — ties last on delivery (2/11), but with the highest ceiling

Delivers only 2/11 — the **same 2** as audit — despite its pipeline *confirming* 6/11 as true-positives internally. Its dedup/trace/report logic (**D8**) discards 4 confirmed corpus matches before the report. It also can't see template files (like audit), and costs the most (\$96, from the deterministic engine doubling task count). It does produce 5 exploit chains. **Its potential recall is real but unrealized as delivered.**

## Fix backlog for VASH (ranked by impact)

- **D8 (highest) — report throws away confirmed matches.** Dedup keeps a canonical finding in a non-matching file, dropping 4 of 6 real corpus hits before the report (delivers 2, confirmed 6). Pure report logic — no re-scan needed. This alone would move VASH's *delivered* recall from 2/11 toward 6/11 and past VVAH — **but that's a claim to be proven by re-scoring the fixed report, not assumed.**
- **D7 — scan template files.** Add `.jinja2` / `.mako` / `.j2` to `find_sinks`; VVAH's unique win (54621) lives there.
- **D6 — cost.** Deterministic engine doubled tasks → \$96 (2× audit). Cap taint/sink-backward counts.
- **D3 — rate-limiting.** Subscription-token throttling stretched wall-clock + failed tasks; an API key would help.
- (Closed: **D1/D2** graph path-mismatch that had disabled taint; **D4** report missing CWE; **D5** false alarm — PoCs do execute.)

## Bottom line (honest)

On **delivered output** — the only fair cross-tool metric — **VVAH wins (4/11), and VASH ties audit for last (2/11)**. VVAH earns it by scanning templates and delivering its matches cleanly. VASH's engine actually *confirms* more (6/11 internally) but a report-filtering bug (D8) discards most of it, so **as it stands today VASH is not better than VVAH on this benchmark**. Whether VASH can pull ahead depends on fixing D8 + D7 and re-scoring — an unproven hypothesis, not a result.

---

Artifacts: `~/vash/results/dmccg-fix/VASH-dmccg-fix-report.pdf` · `~/audit-orig/audit-dmccg-report.pdf` · `~/vul_testing/datamodel-code-generator-0.55.0_report.{md,sarif}` (VVAH).